

Homework 2 — Ingegneria dei Dati

Studente: Michele Guida

Matricola: 576985

Corso di Laurea: Ingegneria Informatica

Anno Accademico: 2025/2026

Repository del progetto: github.com/micheleguidaa/file-search-engine

Dataset

Il dataset impiegato è stato reperito da Kaggle e riguarda ricette di cucina in lingua italiana. In sintesi:

- **Fonte:** [Italian Food Recipes](#).
- **Formato:** CSV (`dataset/recipe.csv`).
- **Contenuto:** collezione di 5 939 ricette italiane.
- **Struttura:** sei colonne (*Nome*, *Categoria*, *Link*, *Persone/Pezzi*, *Ingredienti*, *Steps*).
- **Campi selezionati:** *Nome* (impiegato come nome del file `.txt`) e *Steps* (testo di preparazione, usato come contenuto).

Workflow

L'obiettivo progettuale è l'indicizzazione di un corpus di ricette italiane in Elasticsearch al fine di abilitare ricerche *full-text* efficienti su titolo e contenuto. Il processo si articola in due fasi principali.

1. Estrazione e normalizzazione dei dati

Notebook: `converter.ipynb`

Funzionalità:

- lettura del file `dataset/recipe.csv`;
- estrazione dei campi *Nome* e *Steps* per ogni ricetta;
- generazione di un file `.txt` per ciascuna ricetta nella cartella `files/`;
- normalizzazione dei nomi file (sostituzione di caratteri speciali quali `*` `?` `:`);
- rimozione preventiva di eventuali file preesistenti nella cartella `files/`.

Output: 5 939 file `.txt`, uno per ogni ricetta del dataset.

2. Indicizzazione in Elasticsearch

Notebook: `elasticsearch.ipynb`

Funzionalità:

- creazione (o reset, se presente) dell'indice `index_recipes`;
- definizione del *mapping* con analyzer italiano per i campi `title` e `content`;
- indicizzazione massiva (*bulk*) dei file `.txt` dalla cartella `files/` tramite `helpers.bulk`, riducendo drasticamente il numero di *round-trip* HTTP rispetto all'indicizzazione documento-per-documento;
- misurazione dei tempi di indicizzazione.

Ambiente di esecuzione: Mac M1 con 8 GB di RAM. Elasticsearch è eseguito via Docker Compose (`docker/elasticsearch/docker-compose.yml`) con:

- *single node* (`discovery.type=single-node`);
- sicurezza disattivata per semplicità in locale (`xpack.security.enabled=false`);
- porte esposte: 9200 (HTTP REST API) e 9300 (transport interno).

Analyzer scelti e motivazioni

Nel *mapping* dell'indice `index_recipes`, i campi principali sono definiti come `text` e analizzati mediante l'analyzer italiano predefinito:

- `title: analyzer: "italian", search_analyzer: "italian";`
- `content: analyzer: "italian", search_analyzer: "italian".`

Implementazione dell'analyzer italiano

L'analyzer `italian`, fornito da Elasticsearch e basato su Apache Lucene, è progettato per trattare specificità morfologiche e ortografiche della lingua (elisioni, accenti, plurali, variazioni di genere). La pipeline è sintetizzabile come segue:

```
"tokenizer": "standard",
"filter": [
  "italian_elision",
  "lowercase",
  "italian_stop",
  "italian_stemmer"
]
```

Componenti principali:

- *tokenizer standard*: segmentazione in token e rimozione della punteggiatura;
- *italian_elision*: gestione di articoli e preposizioni elise («l'amico» → «amico»);
- *lowercase*: normalizzazione in minuscolo e insensibilità al *case*;
- *italian_stop*: rimozione di stopwords frequenti («il», «di», «e»...);
- *italian_stemmer*: *light stemming* (Snowball) che preserva distinzioni semantiche: ad es. «mirtillo»/«mirtilli» → «mirtill», mentre «pomodoro» ≠ «pomodorino».

Tale combinazione realizza una normalizzazione morfologica bilanciata, idonea al dominio culinario, minimizzando i falsi positivi; l'impiego simmetrico come `search_analyzer` garantisce coerenza tra indicizzazione e interrogazione.

Numero di file indicizzati e tempi di indicizzazione

Il notebook `elasticsearch.ipynb` implementa l'intera pipeline (creazione indice, *bulk indexing*, cronometria). La misura del tempo complessivo è ottenuta con:

```
start = time.time()
...
end = time.time()
elapsed = end - start
```

Risultati sperimentali

Parametro	Valore
Numero di documenti indicizzati	5 939
Indice di destinazione	<code>index_recipes</code>
Tempo totale di indicizzazione	1.65 s

Discussione. L'uso del *bulk* riduce il numero di chiamate HTTP e sfrutta più efficacemente le risorse del nodo, con sensibili benefici prestazionali. Il tempo osservato (1.65 s) è funzione dell'hardware, della configurazione del cluster e della dimensione media dei documenti.

Query di test e valutazione del sistema

Il notebook `elasticsearch.ipynb` implementa un'interfaccia interattiva per la ricerca, basata su due funzioni principali:

- `parse_and_search()`: traduce query in linguaggio naturale (ad esempio `tiramisu, title: arancini, "burro e salvia"`) in query Elasticsearch strutturate (`match`, `match_phrase`, `multi_match`);
- `search()`: esegue la ricerca e formatta i risultati con highlighting automatico a colori nel terminale.

Tipologie di query supportate

1. **Match** – Ricerca lessicale con analisi morfologica e stemming. Applica l'analyzer italiano configurato, consentendo il match anche su varianti morfologiche (plurali, genere, forme verbali).
2. **Match_phrase** – Ricerca di frase esatta. Richiede che i termini compaiano nell'ordine specificato, preservando la sequenza lessicale originale.

Formato delle query

L'interfaccia accetta query nei seguenti formati:

Ricerca su entrambi i campi (default):

`termine_o_frase`

Esempio: `tiramisu` → ricerca nei campi `title` e `content`.

Ricerca field-specific:

`campo: termine_o_frase`

Esempio: `title: carbonara` → ricerca limitata al campo `title`.

Dove:

- `campo {title, content}`;
- `termine_o_frase` può essere:
 - una o più parole (→ query `match`);
 - una frase tra virgolette (→ query `match_phrase`).

Query di test

Per validare il comportamento dell'analyzer italiano e delle sue componenti nella pipeline di analisi, sono state formulate e testate dieci query, ciascuna progettata per verificare specifiche funzionalità linguistiche e limiti del sistema.

1. Test del filtro lowercase – Case-insensitivity

Query: MOZZARELLA

Obiettivo: verificare che la ricerca sia insensibile alle maiuscole grazie al filtro lowercase, che normalizza tutto il testo in minuscolo.

Risultato: confermata l'insensibilità al case; tutte le varianti vengono correttamente recuperate.

2. Test del filtro italian_elision – Rimozione elisioni

Query: content: l'origano

Obiettivo: valutare la capacità del filtro di rimuovere articoli elisi (es. *l', dell', dall'*).

Risultato: il termine viene correttamente normalizzato in *origano*, consentendo il match con entrambe le forme.

3. Test del filtro italian_stop – Rimozione stopwords

Query: "della panna"

Obiettivo: verificare che le stopwords italiane (es. “della”, “il”, “di”) vengano eliminate durante l'analisi.

Risultato: la parola “della” è ignorata; la ricerca restituisce documenti contenenti “panna”, come previsto.

4. Test dello stemmer – Normalizzazione plurali

Query: content:mirtillo

Obiettivo: controllare che lo stemmer riduca correttamente i plurali alla forma base (*mirtillo/mirtilli* → *mirtill*).

Risultato: la query recupera documenti contenenti sia “mirtillo” che “mirtilli”, dimostrando un corretto comportamento dello stemming.

5. Test dello stemmer – Variazioni di genere

Query: content:fritto

Obiettivo: verificare se lo stemmer gestisce le variazioni di genere e numero (*fritto/fritta/fritti/fritte*).

Risultato: tutte le forme vengono unificate alla stessa radice (*fritt*), consentendo un match coerente.

6. Test dello stemmer – Preservazione semantica

Query: title:pomodorino vs. title:pomodoro

Obiettivo: verificare che lo stemmer leggero (*light_italian*) preservi le distinzioni semantiche tra termini simili ma non equivalenti.

Risultato: le due query restituiscono insiemi distinti, evitando l’over-stemming e preservando la precisione semantica.

7. Test di normalizzazione degli accenti

Query: tiramisu

Obiettivo: controllare che l’analyzer gestisca la normalizzazione degli accenti, permettendo il match indipendentemente dalla presenza di caratteri accentati.

Risultato: documenti con “tiramisu” e “tiramisù” vengono recuperati correttamente.

8. Test match_phrase con elisione

Query: title: "spaghetti all’amatriciana"

Obiettivo: testare la ricerca di frase esatta combinata con la gestione delle elisioni.

Risultato: la frase esatta viene trovata anche in presenza dell’elisione “all’amatriciana”, confermando la coerenza tra *match_phrase* e il filtro *italian_elision*.

9. Limitazioni dello stemmer – Falsi positivi

Query: ciliegina

Obiettivo: evidenziare un caso di over-stemming, dove termini simili (*ciliegina/ciliegino*) vengono ridotti alla stessa radice.

Risultato: la query recupera entrambe le forme, introducendo falsi positivi. Il comportamento è accettabile dato l’approccio conservativo dello stemmer leggero.

10. Limitazioni dello stemmer – Derivazioni morfologiche non gestite

Query: natalizi

Obiettivo: verificare la mancata unificazione di forme derivazionali distanti (*natale* vs. *natalizi*).

Risultato: la ricerca restituisce solo documenti contenenti “natalizi”, non “natale”, confermando che lo stemmer non effettua derivazioni eccessive, mantenendo alta la precisione.

Considerazioni conclusive

I test confermano che l'analyzer italiano di Elasticsearch:

- gestisce efficacemente le variazioni morfologiche di base (plurali, genere, elisioni, accenti);
- rimuove correttamente le stopwords, migliorando la qualità del retrieval;
- preserva distinzioni semantiche rilevanti grazie allo stemming leggero;
- mostra limitazioni prevedibili in casi di derivazioni morfologiche complesse o over-stemming.

Nel complesso, il compromesso tra normalizzazione linguistica e preservazione semantica risulta ben bilanciato per il dominio culinario, in cui la *precisione terminologica* è più rilevante del *recall* assoluto.

Un miglioramento futuro potrebbe prevedere la definizione di un dizionario di sinonimi controllato e la personalizzazione dello stemmer, per ottimizzare ulteriormente il bilanciamento tra precisione e copertura lessicale del sistema di ricerca.